

Informe de Conceptos de Programación

Parte 1. Análisis teórico de conceptos

1.1 Explicación de los conceptos básicos

- **Código fuente:** Es el conjunto de instrucciones escritas por el programador utilizando un lenguaje de programación comprensible para los humanos (por ejemplo, Java o C). Es texto plano.
- **Código objeto:** Es el resultado de traducir el código fuente a lenguaje máquina (ceros y unos) por medio del compilador. Aún no es un programa que se pueda ejecutar por sí solo, ya que le faltan enlaces a librerías u otros archivos.
- **Código ejecutable:** Es el archivo final, listo para ser procesado por el sistema operativo y el procesador. Se genera tras "enlazar" (linkear) el código objeto con las librerías necesarias.

Fases de un programa (Ejemplo con la instrucción `int suma = a + 5;`):

1. **Análisis léxico:** El compilador lee el código y lo divide en "tokens" o palabras clave. En el ejemplo, identifica `int`, `suma`, `=`, `a`, `+`, `5` y `;`.
2. **Análisis sintáctico:** Verifica que los tokens formen una estructura gramatical válida según el lenguaje. Comprueba que después de un tipo de dato y variable, haya una asignación correcta. Genera un Árbol de Sintaxis Abstracta (AST).
3. **Análisis semántico:** Comprueba que la instrucción tenga sentido lógico. Aquí verifica, por ejemplo, que la variable `a` haya sido declarada previamente como un número y no como un texto.
4. **Generación de código intermedio:** Se traduce a una representación independiente de la máquina, más fácil de optimizar.
5. **Optimización:** El compilador intenta hacer el código más rápido o eficiente (por ejemplo, si detecta que `a` siempre vale 2, podría cambiar la instrucción directamente a `suma = 7;`).
6. **Generación de código final:** Se traduce ese código optimizado al lenguaje máquina específico del procesador (Código Objeto).

1.2 Clasificación de lenguajes de programación

Según su nivel de abstracción:

- **Lenguajes de Alto Nivel:** Muy cercanos al lenguaje humano, ocultan la complejidad del hardware.
 - *Ejemplos:* Python (muy legible, usado en IA y web) y Java (usado en aplicaciones empresariales y Android).
- **Lenguajes de Medio Nivel:** Permiten operaciones de alto nivel pero conservan el acceso directo a la memoria y al hardware.
 - *Ejemplos:* C y C++ (ideales para crear sistemas operativos o motores de videojuegos).
- **Lenguajes de Bajo Nivel:** Directamente inteligibles por la máquina o muy cercanos a ella. Dependen totalmente del hardware.
 - *Ejemplos:* Lenguaje Ensamblador (usa mnemotécnicos como ADD o MOV) y Lenguaje Máquina (código binario puro).

Según su paradigma:

- **Paradigma Imperativo:** Se le indica al ordenador *cómo* debe hacer las cosas, detallando paso a paso el control del flujo (bucles, condicionales).
 - *Ejemplos:* C y Python (cuando se usa con bucles for/while clásicos).
- **Paradigma Declarativo:** Se le indica al ordenador *qué* resultado se quiere obtener, sin detallar el paso a paso interno.
 - *Ejemplos:* SQL (se pide una tabla de resultados concreta) y HTML (se describe la estructura de la web, no cómo dibujarla).

Parte 2. Actividad práctica y de análisis

2.1 Identificación de paradigmas de programación a partir de ejemplos

- **Fragmento 1: Imperativo.** Justificación: Se describe el "cómo" paso a paso. Hay un control explícito del flujo indicando que se debe recorrer la lista y sumar uno por uno.
- **Fragmento 2: Declarativo.** Justificación: Se describe el "qué" (empleados mayores de 30 años). Es el motor de la base de datos el que decide cómo buscar internamente sin que el programador defina los bucles.
- **Fragmento 3: Declarativo.** Justificación: Utiliza un enfoque funcional (recursividad matemática). Define qué es un factorial basándose en reglas lógicas y matemáticas, en lugar de explicar cómo iterar sobre los números.
- **Fragmento 4: Imperativo.** Justificación: Explica la mecánica detallada de comprobación (recorrer, comprobar producto a producto, filtrar). Es una instrucción paso a paso del flujo de ejecución.

2.2 Actividad individual: imperativo vs. declarativo

Actividad elegida: Preparar una tortilla francesa.

- **Descripción Imperativa:**
 1. Abre la nevera y saca dos huevos.
 2. Casca los huevos en un plato hondo.
 3. Añade una pizca de sal.
 4. Bate los huevos con un tenedor durante 30 segundos.
 5. Pon una sartén al fuego medio.
 6. Añade unas gotas de aceite y espera a que caliente.
 7. Vierte el huevo batido.
 8. Cuando los bordes estén cuajados, dobla la tortilla por la mitad.
 9. Saca la tortilla al plato.
- **Descripción Declarativa:** Quiero una tortilla francesa de dos huevos, bien cuajada por fuera y jugosa por dentro, servida en un plato.

Comparación: El enfoque imperativo es muy preciso y garantiza que cualquier persona pueda replicar el proceso exacto, pero es largo de escribir y rígido (desventaja). El enfoque declarativo es mucho más rápido, directo y fácil de entender (ventaja), pero delega toda la responsabilidad de ejecución (el "cómo") en la persona o sistema que lo prepara; si no sabe hacer una tortilla, el resultado fallará (desventaja).

Palabra del día: [COMPAÑEROS]